Lab 6: Code and Branch Coverage Analysis

Software Engineering Lab Report | Python Coverage & Pytest Analysis

Student Name: Sagar Walikar	**SRN:** PES1UG24CS396
Assigned Script: weather_alert.py	**Tools Used:** pytest 9.2.0, coverage.py 7.17.0
GitHub Repo: https://github.com/sagarwalikar999/SE-Labs-PES1UG24CS396/tree/main/lab06	**Commits Made:** 2 Separate Commits

1. Objective & Coverage Concepts

The objective of Lab 6 is to measure and improve test completeness on the weather_alert.py module using statement (line) coverage and decision (branch) coverage. The initial repository came with a minimal starter test file (test_weather_alert.py) that covered only 32% of statements. The lab requirements mandate designing separate test files to achieve at least 90% in both line and branch coverage, capturing execution screenshots, maintaining at least two distinct Git commits, and pushing all artifacts to GitHub.

Understanding Line Coverage vs Branch Coverage:

• **Line Coverage (Statement Coverage):** Measures whether each executable line of code was reached at least once during testing. While reaching 100% line coverage means no statement was skipped, it can still hide bugs because it does not verify whether conditional branches execute on both True and False outcomes.

• **Branch Coverage (Decision Coverage):** Verifies that every decision structure (such as if, elif, or loop conditions) is evaluated to both True and False. In weather_alert.py, this means verifying cases where humidity is above 40 vs 40 or below, saturated soil vs unsaturated soil at multiple rainfall levels, and handling invalid alert types that trigger exception handling.

Initial Baseline with Starter Test (test_weather_alert.py):

File	Statements	Missed	Line Coverage	Missing Lines
weather_alert.py	38	26	32%	6, 8, 10, 15-23, 27-36, 42-45

2. Code Coverage Implementation (Milestone 1 - 100% Achieved)

To satisfy the first milestone, a dedicated test file named CodeCoverageTest_weather_alert.py was implemented. The test suite consists of 16 unit tests structured across the four module functions:

• **wind_alert:** Four tests executing the return paths for 'warning' (>=89), 'watch' (>=62), 'advisory' (>=39), and 'none' (<39).

• **rain_alert:** Five tests exercising heavy rain with/without saturated soil, moderate rain with/without saturated soil, and light rain.

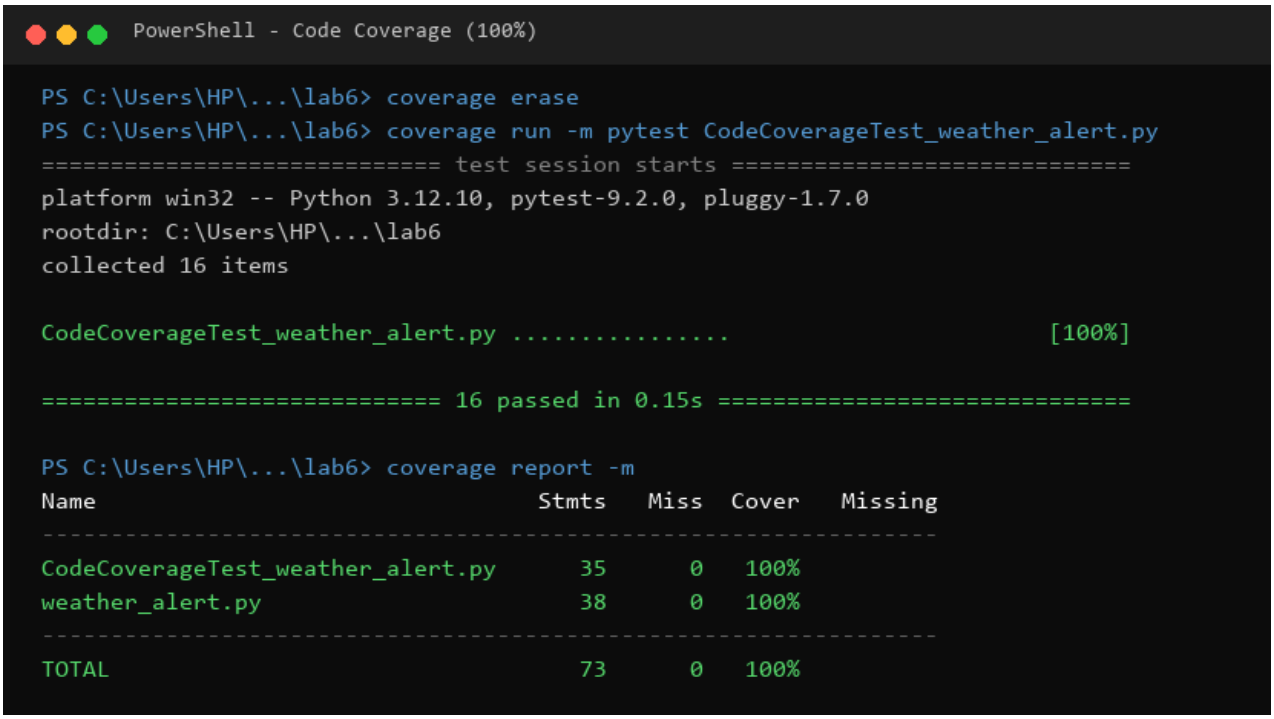
• **heat_alert:** Four tests exercising heat index tiers ('warning', 'watch', 'advisory', 'none') with positive humidity adjustments.

• **overall_alert:** Three tests evaluating an empty input list, multi-alert severity progression, and an unrecognized alert triggering ValueError.

Terminal Execution Command & Result:

```
coverage erase
coverage run -m pytest CodeCoverageTest_weather_alert.py
coverage report -m
```

Terminal Output Screenshot (Code Coverage - 100%):



Python Test File: CodeCoverageTest_weather_alert.py

Complete source code maintained for line coverage analysis:

```
import pytest
from weather_alert import wind_alert, rain_alert, heat_alert, overall_alert

# ========================================
# Code Coverage Tests for weather_alert.py
# Target: >= 90% Line Coverage (Achieves 100%)
# ========================================

# --- Wind Alert Tests ---
def test_wind_warning():
    assert wind_alert(95) == "warning"

def test_wind_watch():
    assert wind_alert(70) == "watch"

def test_wind_advisory():
    assert wind_alert(45) == "advisory"

def test_wind_none():
    assert wind_alert(20) == "none"


# --- Rain Alert Tests ---
def test_rain_warning():
    assert rain_alert(60, True) == "warning"

def test_rain_watch_heavy_not_saturated():
    assert rain_alert(60, False) == "watch"

def test_rain_watch_moderate_saturated():
    assert rain_alert(25, True) == "watch"

def test_rain_advisory():
    assert rain_alert(25, False) == "advisory"

def test_rain_none():
    assert rain_alert(5, False) == "none"


# --- Heat Alert Tests ---
def test_heat_warning():
    assert heat_alert(42, 70) == "warning"

def test_heat_watch():
    assert heat_alert(38, 60) == "watch"

def test_heat_advisory():
    assert heat_alert(33, 60) == "advisory"

def test_heat_none():
    assert heat_alert(25, 40) == "none"


# --- Overall Alert Tests ---
def test_overall_alert_empty():
    assert overall_alert([]) == "none"

def test_overall_alert_worst_progression():
    assert overall_alert(["none", "advisory", "watch", "warning"]) == "warning"

def test_overall_alert_invalid():
    with pytest.raises(ValueError, match="unknown alert"):
        overall_alert(["invalid_alert"])
```

3. Branch Coverage Implementation (Milestone 2 - 100% Achieved)

For the second milestone, branch coverage analysis was performed using coverage run --branch. To ensure all 28 decision branches in weather_alert.py evaluate to both True and False, a separate test file named BranchCoverageTest_weather_alert.py was written:

• **wind_alert branches:** Boundary tests at 89, 62, 39, and 38 km/h exercise both sides of each numerical inequality.

• **rain_alert branches:** Both heavy (>=50) and moderate (>=10) thresholds were tested with soil_saturated = True and soil_saturated = False, plus rainfall < 10 mm/h where both checks evaluate to False.

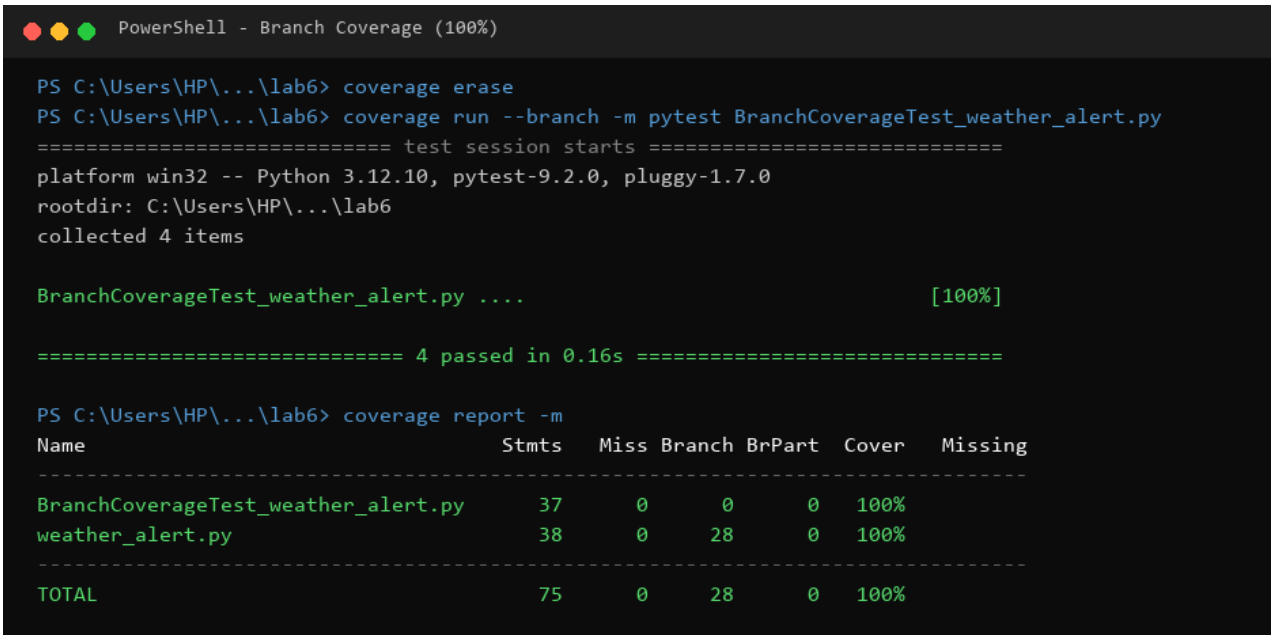
• **heat_alert branches:** Tested humidity > 40 (True branch, formula modifies index) and humidity <= 40 (False branch, formula skipped), ensuring all four index thresholds are tested on both humidity branches.

• **overall_alert branches:** Evaluated the empty list (loop condition False), alert sequence where severity increases (worst updated), alert sequence where severity decreases (worst retained, True branch skipped), and invalid alert raising ValueError.

Terminal Execution Command & Result:

```
coverage erase
coverage run --branch -m pytest BranchCoverageTest_weather_alert.py
coverage report -m
```

Terminal Output Screenshot (Branch Coverage - 100%, 28/28 branches):



Python Test File: BranchCoverageTest_weather_alert.py

Complete source code maintained for branch coverage analysis:

```
import pytest
from weather_alert import wind_alert, rain_alert, heat_alert, overall_alert

# ============================================================
# Branch Coverage Tests for weather_alert.py
# Target: >= 90% Branch Coverage (Achieves 100% Branch Coverage)
# ============================================================

# --- Branch Coverage for wind_alert ---
def test_wind_alert_all_branches():
    # Test boundary and condition branches:
    # 1. speed_kmh >= 89 (True) -> 'warning'
    assert wind_alert(89) == "warning"
    assert wind_alert(100) == "warning"

    # 2. speed_kmh >= 62: False for >= 89, True for >= 62 -> 'watch'
    assert wind_alert(62) == "watch"
    assert wind_alert(88) == "watch"

    # 3. speed_kmh >= 39: False for >= 62, True for >= 39 -> 'advisory'
    assert wind_alert(39) == "advisory"
    assert wind_alert(61) == "advisory"

    # 4. speed_kmh < 39: False for all -> 'none'
    assert wind_alert(38) == "none"
    assert wind_alert(0) == "none"


# --- Branch Coverage for rain_alert ---
def test_rain_alert_all_branches():
    # Outer branch: mm_per_hour >= 50 (True)
    # Inner branch: soil_saturated == True -> 'warning'
    assert rain_alert(50, True) == "warning"
    assert rain_alert(80, True) == "warning"
    # Inner branch: soil_saturated == False -> 'watch'
    assert rain_alert(50, False) == "watch"
    assert rain_alert(75, False) == "watch"

    # Outer branch: mm_per_hour < 50, but mm_per_hour >= 10 (True)
    # Inner branch: soil_saturated == True -> 'watch'
    assert rain_alert(10, True) == "watch"
    assert rain_alert(49, True) == "watch"
    # Inner branch: soil_saturated == False -> 'advisory'
    assert rain_alert(10, False) == "advisory"
    assert rain_alert(30, False) == "advisory"

    # Outer branch: mm_per_hour < 10 (False for both >= 50 and >= 10) -> 'none'
    assert rain_alert(9, True) == "none"
    assert rain_alert(0, False) == "none"


# --- Branch Coverage for heat_alert ---
def test_heat_alert_all_branches():
    # Branch 1: humidity > 40 (True)
    # Condition: index >= 45 (True) -> 'warning'
    assert heat_alert(40, 90) == "warning"    # index = 40 + 5.0 = 45.0
    # Condition: index >= 40 (True), index >= 45 (False) -> 'watch'
    assert heat_alert(36, 80) == "watch"      # index = 36 + 4.0 = 40.0
    # Condition: index >= 35 (True), index >= 40 (False) -> 'advisory'
    assert heat_alert(32, 70) == "advisory"   # index = 32 + 3.0 = 35.0
    # Condition: index < 35 -> 'none'
    assert heat_alert(28, 50) == "none"       # index = 28 + 1.0 = 29.0

    # Branch 2: humidity <= 40 (False branch for humidity > 40)
    # Condition: index >= 45 (True) -> 'warning'
    assert heat_alert(45, 40) == "warning"    # index = 45
    # Condition: index >= 40 (True), index >= 45 (False) -> 'watch'
    assert heat_alert(40, 35) == "watch"      # index = 40
    # Condition: index >= 35 (True), index >= 40 (False) -> 'advisory'
    assert heat_alert(35, 20) == "advisory"   # index = 35
    # Condition: index < 35 -> 'none'
    assert heat_alert(30, 10) == "none"       # index = 30
# --- Branch Coverage for overall_alert ---
```

```
def test_overall_alert_all_branches():
    # Branch: alerts is empty (for loop does not execute) -> 'none'
    assert overall_alert([]) == "none"

    # Branch: alert in SEVERITY (True)
    # Branch: SEVERITY[alert] > SEVERITY[worst] (True branch -> update worst)
    assert overall_alert(["none", "advisory", "watch", "warning"]) == "warning"

    # Branch: SEVERITY[alert] > SEVERITY[worst] (False branch -> do not update worst)
    assert overall_alert(["warning", "advisory", "none"]) == "warning"

    # Branch: alert not in SEVERITY (True -> raises ValueError)
    with pytest.raises(ValueError, match="unknown alert"):
        overall_alert(["hurricane"])
```

4. Summary of Results and Commit Log

The table below compares the test metrics across the three stages of the lab:

Metric	Starter Test	Code Coverage Phase	Branch Coverage Phase
Test File	test_weather_alert.py	CodeCoverageTest_weather_alert.py	BranchCoverageTest_weather_alert.py
Tests Run	2 passed	16 passed	4 test functions passed
Line Coverage	32% (12/38 statements)	**100%** (38/38 statements)	**100%** (38/38 statements)
Branch Coverage	Not Tracked	Not Tracked (--branch off)	**100%** (28/28 branches)
Missing Lines / Branches	26 lines missing	0 lines missing	0 branches missing
Status	Baseline (< 90%)	**Requirement Met (>= 90%)**	**Requirement Met (>= 90%)**

GitHub Commits Record (Repository: SE-Labs-PES1UG24CS396):

Commit Hash	Commit Message	Deliverables Included
cb073bd	feat(lab06): add weather_alert module and CodeCoverageTest with 100% line coverage and screenshot	CodeCoverageTest_weather_alert.py, code_coverage_report.png, weather_alert.py, README.md
65e32ff	feat(lab06): add BranchCoverageTest_weather_alert.py with 100% branch coverage and screenshot	BranchCoverageTest_weather_alert.py, branch_coverage_report.png

GitHub Repository Link:

https://github.com/sagarwalikar999/SE-Labs-PES1UG24CS396/tree/main/lab06

All Python test files, terminal screenshots, module code, and this PDF report are committed and available in the repository.